



ICS 2203 internet application programming

Bsc. mathematics and computer science (Jomo Kenyatta University of Agriculture and Technology)



Scan to open on Studocu

JKUAT

ICS2203: INTERNET APPLICATION PROGRAMMING

ICS 2203: INTERNET APPLICATION PROGRAMMING 1

Course Outline:

Internet Information Architecture. Design and implementation of internet application programs: fundamentals of the WWW. Web browsers and hypertext links. Introduction to client/server architectures. Hypertext Transfer Protocol (HTTP). Internet programming. Hypertext Markup Language. Graphics and images. Video and audio. CGI scripts, JAVA and applets and network programming.

Week	Topic	Subtopics
1 – 2	Internet Information Architecture	☐ Introduction ☐ What is the Internet? ☐ What is WWW? ☐ Fundamental issues and their uses
3	Design and implementation of internet application programs	☐ Introduction ☐ Fundamentals of the WWW ☐ Web browsers and hypertext links ☐ Summary ☐ CAT1 + Assignment1
4 – 5	Introduction to client/server architectures	☐ Introduction ☐ Difference between client and server ☐ How they communicate ☐ Their architectural design ☐ Summary
6 – 7	Hypertext Transfer Protocol (HTTP)	☐ Introduction ☐ What is a protocol ☐ What is http and its various versions? ☐ Its application ☐ Summary ☐ CAT2 + Assignment2
8	Internet programming	☐ Introduction ☐ Use of a scripting language to program ☐ Summary
9	Hypertext Markup Language	☐ Introduction ☐ What is html? ☐ Its application as the programming language for the web ☐ Programming using html ☐ Embedding graphics and images, video and audio ☐ Summary
10 – 11	CGI scripts, JAVA and applets and network programming.	☐ Introduction ☐ What are CGI scripts? ☐ Introduction to Java and applets programming ☐ Concepts of networking ☐ Summary CAT3 + Assignment3
12, 13 & 14	Programming assignment	☐ Applets programming task
15/16	Exams	Exams

Text:

1. *HTML for the World Wide Web* by Elizabeth Castro, Prentice Hall.
2. *Javascript for the World Wide Web* by Tom Negrino and Dori Smith

Some commonly used html tags:

1. center – center text, images and other elements
2. font – changing size, face and color of individual letters or words
3. form – creates fill-in forms
4. frame – creates frames
5. frameset – defines frameset
6. Hn – creates headers, n = 1...6
7. hr – horizontal rules
8. html – identifies a text document as a html document
9. img – inserts images
10. link – used in external stylesheets
11. marquee – creates moving text
12. ol – ordered list
13. ul – unordered list
14. p – new paragraph
15. script – adds automatic scripts to a page
16. select – creates menus in forms
17. sub, sup – subscript, superscript
18. table – creates table
19. tr – table row
20. td – table data/cell
21. title – page title in title bar
22. b, I – bold, italics
23. a – links and anchors
24. !-- - adding comments
25. blink – blinking text
26. body – encloses main section of a page
27. br – line break
28. button – creates buttons

LECTURE 1: INTRODUCTION

This lecture

Overview

This lecture is an introduction to the main concepts and uses of the Internet and the World Wide Web. The purpose of this lecture is to give you a broad overview of the Internet and particularly, the Web.

Introduction to the Internet

What is it?

Before the invention of the Internet, all computers were large, stand-alone single machines which communicated with the outside world through punched cards or punched tape. No computer communicated directly with any other computer (and there were very few computers to talk to anyway!).

The Internet is a loose federation of autonomous computers which communicate via a hierarchy of protocols. The participating computers can be large mainframes, smaller mini-computers (like Sun workstations) or can be personal computers.

To participate in the Internet, a computer (or computer network) needs to:

- connect to another computer on the Internet (with its cooperation), and
- to speak the correct "protocols"

A *protocol* is an agreed set of commands and responses that each side uses according to the agreement, so that they can be mutually intelligible.

Sometimes applications are written which can make computers using different protocols able to understand each other.

However, most computers do not access the Internet directly. Most are part of a sub-network, an *intranet* which in turn accesses the Internet through usually only one or a few machines which are directly connected to the Internet.

This is usually because of security reasons, as it is easier to prevent and detect illegal access from an external source on a few machines than it is on a whole lot of machines. Just one poorly-guarded machine can expose the whole sub-network.

So there is only one Internet, but there are many intranets. And not every intranet is connected to the Internet (again, usually for security reasons).

History

The Internet derives from an earlier computer network called ARPANET. ARPANET was put together in the late 1960s by the Advanced Research Projects Agency (ARPA) of the US Department of Defence (sic).

The story has it that the Dept of Defence needed a computer system that could survive partial destruction.

An alternative story is that the project was intended to create computer communication between its university-based researchers.

Whatever the original incentive, ARPANET and its successor, the Internet, achieve both of these aims very well.

Until the last few years, the Internet was primarily a tool for researchers in universities and government organisations to collaborate and exchange information.

Because there were almost no public Internet Service Providers (ISPs), the only way someone could get onto the Internet was through their workplace, and usually only universities and government organisations had Internet connectivity.

Furthermore, those researchers were almost always scientists, not just computer scientists, but physicists, mathematicians, statisticians, etc. It was the World Wide Web that created the demand for Internet connectivity, and made it easier and more accessible to everyone.

Who is in charge?

Strictly speaking, there is **nobody** in charge of the Internet!

As I said above, it is a "loose federation" of computers and smaller sub-networks of computers.

- there is someone "in control" perhaps for the smaller sub-networks and the individual computers
- however, there is nobody in charge of the whole Internet, which spreads over the entire geographical globe, spanning different countries with their different laws.

However, the Internet mostly runs quite smoothly, despite there being no one body to co-ordinate any necessary repairs, dealing with complaints, etc.

Software companies observe the accepted Internet protocols (such as TCP/IP) because it is in their commercial interest to do so - the Internet has reached "critical mass" and there is a large enough established set of users and software to keep it going as it is.

Who pays?

We do!!!

The Internet is **not** free. It might look that way when you log in to your university account, but in reality, the University has to pay not just for the information being moved (particularly from the USA), but also for the upkeep of its own internal network.

Those of you who have home computers and modems know that Internet connectivity can sometimes be very expensive. The people making the most money out of it are the telephone carriers.

The time spent dialled-up depends on the bandwidth of your connection, the size of what you are downloading and the speed of response from the server. There's not much you can do about the server, but it is worth knowing how much bandwidth some downloadable objects require.

It is possible to get a different form of Internet connectivity depending on the sort of usage.

For example, if you only use email, you would use a "blink" form of connection, where you connect only for as long as it takes to download all your new received emails and send out all the ones you have written.

On the other hand, to access the WWW, you need to be connected continuously. It won't be long before your phone bills start to become very expensive!

S. E. O. Ochingo, SCIT, JKUAT

Soon you will be able to use a new protocol that allocates lots of bandwidth to incoming data and very little to outgoing, which is effectively what happens when you are using the WWW, since most of your requests are very small but data being returned uses a lot of bandwidth.

As the Internet and WWW are becoming more frequently used, so the ways of accessing it are improving.

On the other hand, there is a lot on the Internet that **is** free.

There are many sites and services on the Internet that are provided by their owners just for the public good, and with no expectation of payment. This arises partly because of the Internet's academic origins - people in universities tend to publish the results of their work freely, without trying to sell them. Much of the free software that is available on the Internet was written by computer science students and academics. Telnet and ftp are examples.

The idea of doing things on the Internet for free was so strong until recently that people who tried to advertise things on public bulletin boards were often abused for dirtying the Internet with commercial concerns!

Internet Applications

The main Internet application these days is of course the World Wide Web. Today, we will look at non-Web applications, and afterward we will look at the Web in much more detail.

There are many different ways of requesting information over the Internet:

- Telnet and ftp allow you to retrieve files from a remote machine,
- bulletin boards and newsgroups are a way of publishing information to anyone all
- gophers are a way of finding information about a topic.
- you all know about electronic mail
- virtual places such as chat rooms and MUDs

Now we'll look at these in more detail.

Telnet

Telnet is used to allow people on remote machines on the Internet to log in to another machine. It is a full, live connection to a host machine. It is almost identical to a normal log-on session, except that only text-based data can be used. So for example, you'll usually need to run a text-based mail reading tool.

The earliest version of telnet was developed in 1969, when the Internet itself was being developed.

You can get a *full-privilege* telnet session, which is essentially the same as logging into your account from your normal machine. This requires your normal user ID and password. Also, there is *anonymous* telnet where you are allowed to log in but your activities are restricted.

ftp (file transfer protocol)

ftp means file transfer protocol. This has been around as long as telnet. Basically it is a restricted log-in session with limited access permissions on the host system. Its purpose is to allow people to move files around without having to use email.

This means that binary files such as executables can be moved without needing to be encoded in a special transmission format (such as *uuencode*).

An extreme case of this is anonymous ftp, where anyone at all can log on to the host system and perform a very limited set of requests. The user logs in with user ID "anonymous", and usually are asked to send their complete email address as a password.

e.g. someone makes a file available via anonymous ftp. Anyone who wants a copy of the file can get it using anonymous ftp and the owner need not email it out.

Of course, the access permissions granted to someone connecting with ftp will depend on the user ID they use. You can usually ftp to any machine you have a normal account for and will be given the same read and write access as if you were in a normal log-in session.

ftp has a limited set of commands: *put*, *get*, *cd*, *ls*, and so on.

Bulletin Boards

Bulletin boards and news groups (usually the same thing) are public fora for broadcasting information to a wider audience.

Anyone can *post* (i.e. publish) anything to a news group. It is possible to post a request for information or to post a *follow-up* (i.e. a reply) to someone else's posting.

Many people might all post the same request for information, so people who have not regularly read that newsgroup are advised to seek out the FAQ or Frequently Asked Questions file before posting a request.

There are also a few sad cases who think it amusing to post offensive or stupid messages to news groups. One way to deal with these is to use a *kill file* which automatically filters out any posting from certain named people. Or you can filter out postings according to keywords in the title or content. Another solution is the use of *moderated* newsgroups, where some person acts as a censor, preventing the posting of offensive, boring or irrelevant materials.

People can *subscribe* to a news group so that it is automatically loaded whenever they use a news reader tool. You can do this with Netscape.

Q. Where are news articles stored? Where does the news reading tool go to get a listing of available news articles?

A. Internet-based news groups use broadcast technologies - there is no one canonical site which stores all news articles. There is a news server, an *nntp* server, at most sites, which is responsible for receiving news articles from other sites and for propagating news articles written by locals out to other sites.

Whenever someone posts an article, their local news server propagates it out to n other sites. Those n sites in turn propagate the article out to another n sites each and so on. This is called a *flooding* algorithm and it is *probabilistic*, i.e. there is a high probability that the article will arrive at every news server everywhere in the world, but there is not a guarantee that it will.

Why not transaction-based news propagation? Too much traffic between servers, need for a registry of all participating news servers.

This means that you can post an article and someone at one site can see it immediately but someone at another site sees it much later, perhaps never.

Bulletin boards sometimes use a different approach to Internet-based news groups. Smaller communities of users, such as on the same local network, may have bulletin boards available only to local users. The articles on the bulletin boards may be stored at some site and never need to be propagated to other sites.

Gophers

Gophers are information providing services, rather like text versions of the World Wide Web. The Web uses hypertext links to indicate other information, whereas gophers use menus. Menu items either take you to other menus (perhaps other gophers) or they take you to text files.

Gophers were under development at pretty much the same time that the Web was being developed. However, the hypertext paradigm seems to have come out favourite (perhaps hypertext is a better information retrieval mechanism than menus?) and the Web now has the far greater presence than gophers.

You can actually view gopher information through the Web, it is presented in a menu-style hierarchy in the same way that you view directory structures on the Web.

Electronic mail

You all know about email already (surely...!).

Like telnet and ftp, email has been around since the beginning of the Internet. In just the last ten years or so, email has become one of the most widely-indulged forms of communications after the telephone. There are email-based games, interest groups (mailing lists), friendships and even romances (often with people on another continent!).

Email is a completely different form of writing to anything extant. You no longer write letters where you have to explicitly refer back to your correspondent's previous letter - instead you can include it. That has the following effects

- you have more immediate access to your correspondent's previous communications - you can respond to their exact words in context
- conversations become multi-threaded - all tangents can be followed! There is no longer a monologue, there is an increasingly fractured series of conversations.

What this gives you is something completely different - you can carry on multiple conversations at any time with one person, never losing track of what you are saying because all the history of that thread of conversation is preserved in your email. Even spoken conversations are not like this, because at any one point in time, you can only carry on one thread of conversation, but you usually lose the gist of other threads of conversation in your short-term memory.

The rather conversational style of email tends to encourage people to use it rather informally. This is fine for personal chat but is not always appropriate in the workplace. It is easy to say something informally in an email, but you have to beware that it may be used against you! We use it like speech but tend to forget that once sent, it is permanently on record. In fact, email has been used to prosecute people (more about that in a later lecture).

Another aspect of the informality of email is the tendency to speak freely without realising that words can be ambiguous. It's been estimated that 80% of human communication is non-verbal, so that means with email, we are only getting 20%!

Some mechanisms for expressing non-verbal communication has evolved, and probably the most well-known are *emoticons* such as the smiley face :-) These can be very useful in some situations for removing the ambiguity of plain words, for example, using a smiley face to make it clear that you are making a joke.

Virtual places

MUDs and chat rooms are social applications that use Telnet connections, and sometimes email.

A MUD is a *multi-user dungeon* where a group of people can "get together" in a virtual place (the "dungeon") and act out role-playing games. (MUD got its name from the game "Dungeons and Dragons" which is a paper-and-imagination version role-playing game structure).

What is interesting about MUDs is that they have evolved ways of expressing non-verbal communications, just as email has, and there are commands for "smile", "wave", "laugh" etc. you can even "whisper" to someone in a MUD.

A *chat room* is very similar in principle to a MUD except that it is a virtual place for people to just get together to talk.

📖 You can have public chat rooms (rather like virtual nightclubs) where everyone sees what everyone else is saying.

📖 It is possible to create "private" chat rooms where two (or more, depending on your taste) people can have a chat without others listening in.

In the next lecture

We shall look briefly at the World Wide Web.

LECTURE 2: INTRODUCTION TO THE WORLD WIDE WEB

In this lecture:

- Introduction to the World Wide Web, what it is, who uses it and what for...
- How WWW works - http

N.B. We will be using "World Wide Web", "the Web" and "WWW" to all mean the same thing.

First look at the World Wide Web

What is the World Wide Web?

It is NOT the Internet, though there are many people who confuse the two.

BUT it IS the face of the Internet, i.e. it is the best tool for looking at most of the information available on the Internet.

You might think of the Internet as being the road system and the WWW as being the buses that take you through the road system.

The WWW was "invented" in the late 1980s by two computer support engineers at the European Particle Physics Laboratory in Geneva (CERN). Their motivation was to enable scientists to exchange information across the Internet, using hypertext links. They released their first text-only Web software in 1991.

Hypertext was invented long before this, with the first suggestion of its basic principles published in 1945, and the first implementations in the 1960s.

In 1993, the National Center (sic) for Supercomputing Applications (NCSA) released a graphics-based Web browser, called Mosaic. For a while, this was **the** Web browser, and people would even talk about "Mosaic pages" instead of "Web pages".

In 1994, some members of the NCSA team responsible for developing Mosaic decided that Mosaic was such a hit, they wanted to profit from it. They left NCSA and started up Mosaic Communications Corporation, which was later renamed to Netscape Communications Corporation. They made an awful lot of money out of it...

NCSA stopped developing Mosaic once Netscape dominated the market. This was a drawback in some ways, because they were developing technologies that are **still** not available on commercial browsers, for example, being able to add your own links to someone else's documents.

Having out-competed Mosaic as a browser, Netscape itself is now suffering from the competition of Microsoft's Internet Explorer (IE). Netscape are trying to salvage their market share by concentrating on servers and by distributing their browser, including source code, free of charge (not that this worked for Mosaic). Also Microsoft is facing "anti-trust" legal action, because of their attempt to completely wipe out Netscape from the browser market - this is ongoing.

How WWW works

Overview

The Web is really a huge collection of data, much of it in HTML format, stored at lots of different computers on the

Internet.

WWW uses the client/server approach. There are a number of autonomous servers and a larger number of clients, called *browsers*. Browsers are used for looking at the data on the Web - a browser sends a request to a Web server for a specified document and the server sends that document back.

The preferred format for browsers is *HyperText Markup Language* (HTML). It was designed as an "interchange format", which means that it is designed to be not specific to any particular machine, and uses only ASCII characters. All Web browsers should be able to correctly process an HTML document (although some browser developers like to develop their own special tags which don't work very well on any browser other than their own (Netscape were prone to this when they first went public).

The primary means for a browser to request information from a server is to use *hypertext links*. Hypertext links contain Uniform Resource Locators (URLs) which are embedded http requests usually. These are sent to the server named in the URL (perhaps via a proxy) which then locates the file named and sends it back. The http request contains all the information needed to locate the appropriate server and to tell that server what to send back. So why would anyone put up information on the Web? Who wants to set up and manage servers, and why?

The WWW as an Academic Resource

The original purpose of the WWW was to enable scientists to exchange information.

So the WWW was originally developed as an academic resource.

Today, the WWW is even stronger as an academic resource, with more academic publications being duplicated on the WWW, and more researchers putting copies of their work on their personal and laboratory pages.

Not so long ago, to get a copy of a research paper, it could take weeks to either get it via inter-library loan or by asking the author to send a copy.

In contrast, publications on the WWW can be accessed within minutes.

So, more information is being made available, and faster. This makes it harder to keep up with what is happening!

Commerce on the WWW

More and more commercial companies are advertising goods, services and information on the Internet, using the WWW.

Even jobs are being advertised on the WWW and because information access is so rapid, people from anywhere can get the information within minutes, instead of weeks.

Some companies only make information available but others will take purchases ordered via the WWW.

Companies only publishing information will often make a profit by the advertising they carry on those information pages. Search engines like Yahoo! carry a lot of advertising.

Other companies make a profit by charging slightly more for their information services (e.g. classified advertisements listings).

The Web's publishing model

S. E. O. Ochingo, SCIT, JKUAT

Anyone anywhere can put up anything at all on the Web. All you need is file space on a Web server and enough knowledge to make an HTML file (or even just a plain text file).

Unlike the current publishing media we already know, such as books and magazines, television and radio, there is no "editor" who will approve or reject material for publication. That means that it is possible to put up material that has not been "vetted", so the quality of what you read could be completely unknown.

- Sometimes you can assess the quality of something you read according to who provides it. For example, you would expect something at a Government Web site to be reliable in what it says. But this is not always the case! There are many documented cases of official Web sites being hacked and the original, approved contents being replaced with something that the owners wouldn't approve ever. That even includes the UK and USA governments:

So would you trust everything that you read on a Web site?

Mind you, most "hacked" Web pages are pretty obviously hacked. But it is just as possible to create more subtly hacked pages that can cause not just embarrassment, but which deliberately mislead the reader. For example, a competitor could put up falsely inflated prices on someone's on-line sales pages, in order to frighten off potential buyers - this is a form of *industrial espionage*. But perhaps the nastiest form of attack is what can be called *information warfare* where attacks are made on servers with the deliberate intention of destroying the service, such as with last year's attack on the East Timorese domain, .tp, which was blamed on the Indonesian government.

On the other hand, where the material is reliable, it can be a brilliant source of useful material. The vast majority of Web sites are well-maintained, and hacked pages tend not to stay up for very long, especially when someone who knows the real page alerts the owners. Also, some pages change so often, it is hardly worth the effort for hackers to change it. For example, there's not much point in hacking the Guardian online, when they change the contents so often anyway.

Another drawback of the Web free-for-all publishing model is that of *obscurity* - it might be that nobody will find a page you put up. It depends who you are - for example you can guess a Web page address for a company if you want information on them. If it is a commercial company try "www", then the company name, then "com" or maybe "co.uk" or "com.au". However, if you are just wanting to put up your own information, you really need to draw attention to your site some other way, such as emailing a news group or a search engine manager.

http

The *HyperText Transfer Protocol*, or *http* for short, is the language, or "protocol" that Web clients (browsers) and Web servers use for communicating. It is fundamental to almost every Web request.

Recall that a *protocol* is an agreed set of commands (requests) to be supported and responses to those commands.

WWW uses *http* - hypertext transfer protocol - as its main protocol, although it also supports *ftp* and some other protocols.

How http works

All http transactions have the same general format. Every client request and every server response has three parts: the request or response line; a header section; and the entity body.

A client initiates a transaction like this:

1. The client contacts the server, and sends a document request by specifying a http command called a *method*, followed by a document address and an http version number.

2. After this, the client might optionally send some header information to give the server information about the client's configuration, and what document formats it can accept (e.g. GIF, JPEG).
3. After sending the request and headers, the client can optionally send additional data. This is mostly used for CGI programs, although some browsers can use it to publish an edited page back onto the Web server.

A server responds to the client's request like this:

1. The server replies with a status line which contains information about the http version, the status code and description. The HTTP version is the one that the server is using to respond. The status code informs the client of the result of the request, and the description is just a human-readable description of the result. For example, a successful retrieval of a document will return status code 200 and description "OK". Another example is when a document is not found and the status code and description are "Error 404: document not found".
2. After the status line, the server sends header information to the client telling the client about the server itself and the requested document, such as the server software name, the document type and when it was last modified, and maybe how long it is.
3. If the client's request is successful, then the last part of the server's response is the actual document or data requested. This might be a copy of a file, or it could be the result of a CGI program execution. Alternatively, if the client request failed, the server might send some additional information for the user to read about why the request could not be fulfilled.

http methods

As noted above, the client specifies a *method* within the document request sent to a server.

A method is a way of instructing the server how to respond. The `GET` method is used to retrieve a document stored by a browser (this information could be a stored document or a CGI script result). The `HEAD` method is like `GET` except that only the header information for the requested document is wanted. `POST` allows the client to send some parameters for other programs the server accesses, such as the news server or a CGI script.

There are other methods which are used for associating or dissociating header information with a document the server (`LINK` and `UNLINK`) or for debugging (`TRACE`) or for writing or deleting information to the server (`PUT` and `DELETE`).

State (or otherwise!) in http

In http version 1.0, after the server has completed the client's request, it disconnects itself from the client. So you do not 'log on', perform one or more operations, then log off. The http server does not remember anything between the client's requests. The client has to establish a new connection every time a request is to be made.

e.g. loading a document often involves loading images - separate http requests must be made for each image in addition to the original request for the whole document. The only difference is that the browser automatically requests images whereas the user most likely requested the original document.

In http version 1.1, the server by default will keep the connection open, so that additional requests can be made. This allows the client to request not just the original document, but any images, frames and applets that are associated with the original document. Usually the connection remains open until either client or server explicitly closes the connection.

However, http is a *stateless* protocol. This means that the server does not keep a record of what the client did previously. This means that every client request has to include all the necessary information as if the connection was just being opened.

S. E. O. Ochingo, SCIT, JKUAT

Telnet and ftp are both *stateful*, because you log on for a session, previous commands being remembered and influencing subsequent commands (e.g. the change directory command).

The advantage in http being stateless is that it can serve a lot more clients because it does not incur any overheads for tracking client sessions between connections. The disadvantage is that when it is necessary to maintain state between sessions, other tricks such as cookies are needed.

Preserving state (and invading privacy?) with cookies

Client side cookies were introduced by Netscape to enable the server to record information about the client so that subsequent requests can be "better" serviced and state can be preserved between transactions. The server can also tailor the response to the client using the information in the cookies file about the last visit to that server.

Cookies actually work by allowing the server to write information to the client's machine. A cookie is created when a client visits a site for the first time, and stores this in a "cookies" file on the client's machine. Subsequently, when a client goes to that site again, the client checks its cookies file for any information (cookies) about that URL. If there is any such information, the client includes a `cookie` header in its request to the server.

There are drawbacks with cookies - they can be misused in a number of ways. In the first place, it is not always a good idea to let someone write something to your hard disk if they are not a known and trusted source.

A more insidious use of cookies is to gather information about the user. It might be nice to have advertising targeted at you tailored to your own interests. On the other hand, not everyone like the thought of Big Brother watching them, a server analysing their every move through its Web site and recording information about what they look at, how often, how much they spend on certain things.

The information servers can gather about your reading and spending habits can be sold on to information brokers and marketing companies. And they don't always have the courtesy to tell you they are doing this. Some browsers can be set to refuse all cookie information, but you can always make your cookie file read-only, which seems to work just as well!

LECTURE 3: HTML BUILDING BLOCKS

HTML documents can be created with any word processor or text editor. Examples include Notepad or WordPad for Windows and SimpleText or TeachText for Mac.

Also Web page editors exist for ease of creating Web pages. Some examples of editors include PageMill or DreamWeaver.

HTML tags:

These are normally commands written between "<" and ">" signs.

In most cases there are opening and closing versions of such tags where a closing tag is preceded by "/" symbol.

An example is `<b>some text</b>`

Attributes:

These are some features contained within tags offering a variety of options for any contained text. An example is `<table border>` in this case, border is an attribute.

Values:

These are features assigned to attributes e.g. <br clear = left> where “left” is the value for the “clear” attribute.

Quotation marks:

Always get accustomed to enclosing your values in quotes. This is strictly not necessary in some cases, for example when there is only one item for the value.

An example is <br clear = left>

Upper and Lowercase letters:

Case sensitivity is not important in creating HTML documents, e.g.
 is similar to
.

Nesting tags:

This is a case of affecting your text with more than one tag. The order should be First tag ends Last. e.g. <i>some text</i> and not <i>some text</i>

Spacing:

HTML browsers ignore any spaces existing between the tags in HTML documents. Also, pressing Returns and spaces to format your text offers no help. You must only use the required tags for this purpose.

Note:

Some tags, known as Block-level tags offer automatic line breaks e.g. P,H1,BR,UL,TABLE.

Others, termed inline tags that only affect letters or words do not automatically begin as a new line e.g. EM, B, IMG.

Special symbols:

These are symbols that may not be available on your keyboard. To use them, use a name or number code for it. (refer to the recommended text pages 355-356).

File names:

Good file naming will help you organize your files, make it easier for your visitors to find and access your pages and ensure that their browsers view the page correctly.

It is therefore a good idea to use only lowercase letters for file names.

For file name extensions, use .htm or .html. A file name extension such as .txt will cause the browser to show even your html codes.

Uniform Resource Locators:

This is the equivalent of an address. It indicates where a file is and what a browser should do with it. An example is <http://www.maseno.com/sam/file1.html>

- http:// is the scheme.
- www.maseno.com is the server.
- /sam/ is the path.
- File1.html is the file name.

Absolute and Relative URLs:

URLs can either be absolute or relative.

An absolute URL shows the entire path to a file i.e. the scheme, server, path and file name, just like the complete address of some place.

Relative URLs simply describe the location of the file that contains the URL itself e.g.

“images/image.gif”

Advantages of relative URLs:

- being much easier to type thus making it easier to move your pages from a local system to a server.
- you do not have to type the scheme as long as it is http.

LECTURE 4: STARTING YOUR WEB PAGE

This teaches you how to:-

- design and organize your page
- give your page structure
- begin writing your page contents
- Save and look your page

Designing your site:

Consider the following when undertaking this activity:-

- Figure out why you recreating the page, i.e. what exactly are you conveying.
- Think about your audience e.g. your page should be appealing and load quickly
- Depending on the number of your pages, choose the best way to go about them
- You can pre-sketch on paper your layout and device a consistent way of naming your files, images etc.

Note: Your files may be organized in the same or different folders. In case of different folders, you must qualify the path accordingly.

Starting the Web Page:

The simplest structure you need is:

```
<HTML>
<HEAD>
<TITLE>
```

(Insert some title here. It appears in the title bar of the window. The title must be short and descriptive enough)

```
</TITLE>
</HEAD>
<BODY>
```

(Insert the body here. It contains the content of your page)

```
</BODY>
</HTML>
```

Note: The HEAD and BODY tags are optional but important in ensuring that:

- They show where they begin and end. This is important in that there are some items that strictly need to go to the HEAD section only.
- They control when a particular script will run (e.g. when you include JavaScript in your pages).

But the TITLE tag is a must. Note that the TITLE is used by search indexes like YAHOO and ALTA VISTA as well as in your visitors' browsers' history lists and bookmarks.

Organizing a Page:

HTML offers six levels of headers to help in organizing your pages namely h1, h2,...h6. h1 has the biggest font size while h6 the smallest. To use them simply do the following:

```
<h1> some text here </h1>
```

Starting a new paragraph:

This is achieved by the use of the <p> tag. Do the following

<p> some text here </p>

However note that the closing tag is not mandatory.

Saving and Viewing your page:

After all that is done, save your work as a text file with a .html or .htm extension.

From here, use any browser to view it. For example, go to IE or Netscape, choose File/Open/Browse to the correct folder/ choose the file and open it.

LECTURE 5: TEXT FORMATTING

Text formatting refers to the process of modifying your document so that it achieves the kind of look you best desire.

This is done by enclosing the desired text in tags that indicate the formatting style you wish to apply.

Note that all formatting in HTML is contained in text thus your pages can be read by any computer.

There are two methods to achieve formatting.

- The first uses regular HTML tags and attributes to format distinct pieces of text individually.
- The second, which applies if your formatting is complex or your documents are long and numerous uses “Style sheets.”

Let us look at the first case (Second case will come later).

Changing Font: You can choose the type of font you would want to use in your web pages. However, note that if your client does not have those fonts the text will be rendered in the default font.

The syntax is `<font face = “fontname1, fontname2,..fontnamen”> </font>`

An example is

```
<html>
<head>
<title>using different fonts</title>
</head>
<body>
```

`<Font face = “lithos black, Chicago”> You can change the font face </font>` of just a few letters

```
</body></html>
```

Making Text Bold or Italics:

Use tag `<b>` for bold and tag `<i>` for italics as shown below.

`<b> some text here</b>` this causes the text in between to be bold as shown “**some text here.**”

`<i> some text here</i>` this causes the text in between to be italics a shown “*some text here.*”

Choosing a Default Size for Text and Changing Text Size:

You can select a size for all of the body text on your page with the BASEFONT tag. However, other individual sections may be changed as you wish with the FONT tag.

For example:

```
<html>
<head>
<tittle>using different fonts</title>
</head>
<body>
```

`<basefont size = “n”>` where n is some number e.g. 4

```
</body>
</html>
```

In order to change the text size in other sections of your page use the FONT tag as shown below:

```
<font size = "n"> where n is some number</font>.
```

Note that the BASEFONT applies to the whole page while the FONT tag applies to only text between it.

Choosing Default Text Color and Changing Text Color:

Just like dealing with text size above, the TEXT attribute for the BODY tag lets you specify a default color for all the text on your page. However, you can change the text color in other sections by using the FONT tag.

The Syntax is

```
<body text = "color"> where color represents the color you want.
```

The color can either be in hexadecimal representation eg. #rrggbb or in one of the 16 predefined colors.

However, in order to change text color in other sections use the syntax below:

```
<font color= "color"> I love this color</font> where "color" represents the preferred color.
```

Creating Superscripts, Subscripts, Striking out and Underlining Text:

Use the <sub> tag or <sup> tag to create subscripts and superscripts respectively.
For example

today ₂ displays as today₂. While today ² displays as today².

Use the <STRIKE> or the <U> tags to cause striking out or Underlining for your text as shown below:

```
<strike> This is stricken text </strike> displays as This is stricken text
While <U>applied in the same context causes underlining.
```

Blinking Text and Hidden Text (Adding Comments):

You can make text on your page to blink. This is especially useful to attract the attention of your clients. This can be achieved by the <BLINK> tag as shown below:

```
<blink> This text is blinking </blink>
```

Usually the use of comments is quite important in programming. In HTML also this is useful in reminding you of what you were trying to achieve with your HTML tags. These comments appear only in the HTML document but not when viewed by a client. In order to add comment on your page use the syntax below:

```
<!-- This is a comment line -->
```

Note: the "text" attribute was deprecated in HTML 4 so work with styles.

LECTURE 6: USING IMAGES

Inserting an image on a page:

All kinds of images can be placed on your Web pages, from logos to photographs. They then automatically appear on your browser when viewed.

Use the following tag:

`<IMG SRC= "img" BORDER = n>` where n is a number.

Offering alternate text:

This is important in the case of browsers that are unable to show images. Do the following:

`<img src = "img" alt = "the text">`

Linking icons to external images:

For large images that may take time to load, you may use its miniature version for faster viewing and create a link to the larger version. It is done thus:

`<a href = "large image"><img src = "iconimage" alt = "the text"></a>`

Using low resolution images:

This causes the low-resolution image to load faster as the high-resolution image appears slowly after. This is important in keeping your visitor's patience instead of waiting for too long for the high-resolution image to appear. Do this:

`<img src = "image" lowsrc = "imagelow" height =x width = y alt = "the text">`

Wrapping text around images:

This is achieved by the use of **ALIGN** tag plus the attributes **left** or **right**.

`<img src= "image" align = left>`

This causes the image to appear on the left while the text wraps on its right. If you use right, the image appears on the right and the text wraps from the left.

Stopping text wrap:

Use the BR tag with the CLEAR attribute to achieve this. The cursor must be, in this case, at the place where you wish to stop the text wrap.

`<br clear = left>` for the left case, or `<br clear = right>` for the right case or `<br clear = all>` for both cases.

Adding spaces around images:

This is a way of keeping away your text from being so close to your image. It is achieved by the application of the attributes VSPACE and HSPACE.

`<img src = "image" hspace = x vspace = y>` where x and y are numbers in pixels.

Scaling an image:

This is a way of reducing the size of your image for ease of loading. Too much of enlargement of your pictures though causes them to be grainy and ugly. The smaller the image, the faster it loads and thus does not keep your visitors waiting for long.

`<img src = "image" width = x height = y>` where x and y are in pixels.

Aligning images:

There are various ways of aligning your image with the text in your document i.e. on a single line in your paragraph. Available options are texttop, top, middle, absmiddle, bottom, or absbottom. An example is

`<img src = "image" align = direction>` where direction is one of the attributes mentioned above.

Adding horizontal rules:

This is achieved by the use of HR tag as below

`<hr size = n width = w align = direction >` where n is height of rule in pixels, w is the width and align can be left, center or right.

LECTURE 7: PAGE LAYOUT

In this lecture, we learn about tags that apply to an entire page instead of just affecting a few words or a few paragraphs. These include features such as setting margins and columns, changing background color etc.

Using background color:

Use the BGCOLOR tag to change the background color of your pages as below

`<body bgcolor = "color">` where **color** is the desired color.

Using background images:

In the BODY tag, do the following

`<body background = "image">` where image is the desired image. Note however that smaller images will be automatically tiled by browsers.

Centering elements on a page:

Use the CENTER tag as follows

`<center>` the text `</center>`

Specifying margins:

This differs for IE and Netscape as follows

IE: `<body leftmargin=x topmargin=y>`

Netscape: `<body marginwidth=x, marginheight=y>`

To create a new line of text, use the BR tag as below

`<br>` the text

Use of multiple BR tags creates extra space between lines of paragraphs.

To Keep lines together, use the NOBR. This can be like in the case where you do not want a certain phrase separated.

`<nobr>` the text `</nobr>`

However the WBR tag, known as the discretionary line break, ensures that line breaks do occur only when it is absolutely necessary.

Specifying space between paragraphs:

Netscape achieves this by use of SPACER tag as below

`<spacer type = vertical size = n>` where **n** is in pixels.

To create indents, do this

`<spacer type = horizontal size = n>` where **n** is as previously said.

Indents can as well be created by the use of lists as below,

- Type the paragraph
- Type the text

This should work well for Netscape.

Creating blocks of space

The SPACER tag can be used to create blocks of space around which text can be wrapped as below (works with Netscape only)

<spacer type="block" width=w height=h align= direction> where direction is left or right.

If you use top, middle or bottom as direction, text will not be wrapped around the block.

Using block quotes:

Block quotes set off a section of your text from the surrounding text. This can be like a case of quotations from a famous author. Some browsers will center it while others will italicize it i.e. the block quote.

<blockquote> the text </blockquote>

eg

The above is for block-level quotes, ie quotes that start with a new paragraph.

For inline quotes, eg quoting short passages of text, do the following

<q lang = xx> the text </q> where xx represents the two letter code for the language the quote will be in.

Note: the above tag is currently not supported by the common browsers but may be useful in the future.

Creating columns:

Only Netscape recognizes this feature.

<multicol cols = n gutter = x width = y> the text </multicol> where n is number of columns, x is space between columns and y is the width of the entire column i.e. including the gutter.

Using preformatted text:

Preformatted text allows you to maintain the original line breaks and spacing that you have inserted in your text. This is because normally it is the browser that decides where to divide each line of text, depending on the size of the window, and eliminates extra spaces and returns. To use it, apply the PRE tag as below

<pre> the text with all the necessary spaces, returns and line breaks.</pre>

Positioning elements with layers (Netscape only!)

This allows you to lay your html documents in layers. The syntax is

<layer id = name top=m left=n width=w height=h src="source.html">. There are more attributes that may be added.

- name = identifies the layer
- m in pixels the offset of contents from top
- n in pixels the offset of contents from left
- w,h = width and height respectively
- src = is the source html

An example may look thus:

```
<layer id = layer1 top=10 left=50 width=100 height=100 bgcolor=green>
```

LECTURE 8: LINKS

In this lecture, we shall learn the various ways of creating and manipulating *links*. Links let you skip from one page to another, call up a movie, download files with FTP etc.

A link has basically three parts

- Destination – used to specify what will happen when the visitor clicks the link.
- Label – the part the visitor sees and clicks to reach the destination
- Target – usually ignored or left to the browser, determines where the destination will be displayed.

Creating a link to another Web page:

Use the **A HREF** tag to achieve this as below

`<a href = "page"> label text here </a>` where *page* is the URL of the page and *label* is the clickable text.

e.g. `<a href = "http://www.site.maseno.com/csd/rules1.html">department rules</a>`.

Creating Anchors:

A click on a link normally takes you to the top of a page. Anchors instead take you to specific sections of a page. The procedure is

- Place cursor in the part of page you wish user to jump to
- Type `<a name = "anchor name">` where anchor name is the text you will use internally to identify that section of the Web page.
- Add the words or images that you wish to be referenced
- Type `</a>`

(see page 120 of recommended text for an example)

Linking to a specific anchor:

Once an anchor is created, you can define a link so that a user's click brings them directly to the desired section as below

`<a href = "#anchor name"> text to be clicked </a>`. The "text to be clicked" refers to the words or images to be referenced.

Note:

If the anchor happens to be in a separate document, use

`<a href = "page.html#anchortext">`, i.e. provide the absolute path.

If the anchor is on a different page on a different server, use

`<a href = "http://www.site.com/path/page.html#anchortext">` in order to reference the whole path i.e. the url.

Creating other kinds of links:

The creation of links is not limited to only Web pages, you can do much more.

- To create links to files on the Web – `<a href = "http://www.site.com/path/filename">`
- To create links to ftp site – `<a href = "ftp://ftp.site.com/path">`
- To create links to an email address – `<a href = "mailto:name@site.com">`
- To create links to telnet – `<a href = "telnet://site">` where site is the server name you want the connection to etc.

Creating key board shortcuts for links:

To add a key board shortcut to a link, do thus

```
<a href = "woody.html" accesskey = c> Cookie</a>(alt+c)
```

Note: Keyboard shortcuts do not work in Netscape.

Using images to label links:

You can add an image to a link to create a navigational button that the visitor clicks to access the referenced URL as shown below.

```
<a href = "destination.html"><img src = "image" border =n></a> where  $n$  is number in pixels for the border.
```

Changing the link colors:

Using the LINK tag lets you change the colors as below:

Inside the BODY tag,

- LINK changes the color of links that have not been visited.
- VLINK changes the color for links already visited
- ALINK changes colors for links when user clicks on it.

An example:

```
<body alink = "#242424" vlink = "#616161" link = "#bababa">
```

The above line causes such effects on your document.

LECTURE 9: LISTS

It is possible to create plain, numbered or bulleted lists. It is also possible to nest one kind of list inside another.

Creating ordered lists:

This can be created anywhere in the BODY section of your page. They are good in explaining step-by-step instructions for how to complete a particular task or for creating an outline of a larger document.

The general syntax is

```
<OL TYPE = X, START = m,> <LI TYPE= Y, VALUE = n> </OL>
```

where X = symbol to use e.g. “A” for capital letters, “a” for small letters, “I” or “i” for capital or small roman numbers, and “1” for numbers by default.

Y = kind of symbol to use and overrides “X”, “n” overrides “m”.

Example:

```
<ol>
<li>My first line
<li>My second line
<li>My third line
</ol>
```

Creating unordered lists:

These are used to list any items that fall in no particular order. The general syntax is

```
<UL TYPE = shape> <LI TYPE = shape> </UL> where shape can be disc, circle/round or square.
```

An example:

```
<ul>
<li type = round> this is rounded line
<li type = disc> this is disc line
<li type= square> this is square line
</ul>
```

Creating definition lists:

This type of listing is particularly suited to glossaries or any list that pairs a word or phrase with a longer description. The procedure is

- Type the introductory text
- Type <DL> (*definition list*)
- Type <DT>, type the word/phrase to be explained. (*definition term*)
- Type<DD>, type the definition of the word/phrase above (*definition data*)
- Repeat steps 2 and 3 above
- Type </DL>

An example may be found on pg 140 of the recommended text.

Creating nested lists:

Allows for the insertion of one list into another, and is useful with an outline rendered with ordered lists, where you may want several levels of items.

An example:

```
<h1> The great American Novel</h1>
<ol type = I>
<li>Introduction
    <ol type = A>
        <li>Boy's childhood
        <li>Girl's childhood
    </ol>
</li>
<li>Development
    <ol type = A>
        <li>Boy meets girl
        <li>Boy and girl fall in love
        <li>Boy and girl have fight
    </ol>
</li>
</ol>
```

LECTURE 10: TABLES

Tables are made up of rows of cells and its shape is determined by the number of cells in each row.

Creating a simple table

In the body part of your page, do the following:

```
<TABLE>
<TR>
<TD>
Type the contents of the cell here
</TD>
</TR>
Repeat the steps in between <TR> and </TR> as desired
</TABLE>
```

where

<TABLE> – represents opening table tag.

<TR> - represents table row.

<TD>- represents table data.

Adding table border

A border helps separate your table from the rest of the text.

To set it, inside the initial TABLE tag, include

BORDER = n, where “n” is a number in pixels representing the thickness of the border.

Changing the border color

This is applicable for IE only. Usually border color comes from the color of the background.

Inside the TABLE tag, type

BORDERCOLOR = “color” where color is the preferred color.

Setting the width of cell or table

Browsers automatically determine the width of your table by calculating the width of the elements and text it contains.

In the respective TD or TABLE tags, type

WIDTH = “n” where n is a value in pixels.

ie <table width = 320 border = 0>

 <tr>

 <td width = 320>

Centering table on a page

In the table tag, type **ALIGN = center**

Note that you can also center a table by enclosing it within CENTER tags.

Wrapping text around a table

This can be achieved as in the case of images. To wrap text, within the TABLE tag, type ALIGN= left or right.

To stop text wrap, simply use the <BR CLEAR=left> or right as is appropriate.

Adding space around a table (for Netscape only)

This causes the table to stand out on a page. To add the space, type inside the TABLE tag

HSPACE= h for space on the right and left or

VSPACE=v for space on the top and bottom of the page.

Spanning a cell across columns or rows

At the point where you wish to span a cell across more than one column or row, type

<TD COLSPAN = n>

type the cell's contents

</TD>

Where n is a numeric value and is the number of columns the cell should span. eg <td colspan = 2>

Use ROWSPAN instead of COLSPAN in the case of spanning across more than one row.

Aligning a cell's contents

In the initial tag for the cell, row or section, type

ALIGN = direction, where direction is left, center or right.

VALIGN = direction, where direction is top, middle, bottom or baseline.

Controlling space in and around cells

Cell spacing adds space between cells, makes table bigger but does not affect the size of individual cells.

Cell padding adds space around the contents of the cell, thus pushing the walls of the cell out.

In the TABLE tag, type

CELLSPACING =n

CELLPADDING = n

Where n is a number in pixels.

eg <table width = 480 cellpadding = 4 border = 0>

<tr><td colspan = 3 valign = bottom width = 480>....</tr>....

Nesting one table in another

This is possible by

- Creating the inner table
- Create the outer table and determine which cell of the outer table will contain the inner table.
- Test both tables separately
- Insert the inner table into the correct cell.

Changing a cell's color

Within the TD tag type

BGCOLOR= “color” where color is the preferred color

Note: You can use background images for cells or a whole table.

For a cell, type,

In the TD tag, **BACKGROUND = “image.gif”**

For the entire table, in the TABLE tag, type **BACKGROUND = “image.gif”**

LECTURE 11: FRAMES

Frames help in organizing your site for easy navigation. It allows your visitors to have an idea of the scope of information contained in your site and then making that information easily accessible without confusing or overwhelming them.

Your page, the *frameset*, will be divided into *frames*, each containing a Web page and could be viewed independently in a separate window.

A frameset is like a window with individual panes, each pane showing different information.

Creating a simple frameset

After the </HEAD> section, type

```
<FRAMESET ROWS = "A, b">
```

```
<FRAME NAME = "name" SRC = "content.html">
```

repeat the above line for each frame you wish to create.

```
</FRAMESET>
```

where

a = height of first row in %, exact number or *, ie variable.

b = height of second row and similar to a in characteristics.

FRAME= to assign a URL and other attributes to the top row/pane.

name= is a name to identify this particular frame's use.

content.html = URL for the page that will be initially displayed in this frame when the visitor first navigates to this frame.

Here is an example:

```
<html><head><title>Frames in rows</title></head>
```

```
<frameset rows="65,*,60">
```

```
<frame name="banner" src="banner.html">
```

```
<frame name="photos" src="openingpage.html">
```

```
<frame name="buttons" src="buttons.html">
```

```
</frameset>
```

Creating frames in columns or rows

After the </HEAD> tag, type

```
<FRAMESET COLS= "a,b">
```

```
<FRAME NAME= "name" SRC= "content.html">
```

repeat the above step for each frame/column

```
</FRAMESET>
```

To have them in rows, simply replace COLS with ROWS.

Creating frames in Rows and Columns

If you wish to have frames in both columns and rows together, follow this,

```
<FRAMESET
```

```
ROWS= "a,b"
```

```
COLS= "x,y">
```

```
FRAME NAME= "name" SRC= "initialurl.html">
```

Repeat the above statement for each row from top to bottom.

```
</FRAMESET>
```

Combining Framesets

You may want, for example, to create one row at the top of your site that spans the width of your browser while the second row is divided into frames. This is possible by inserting a frameset in the second row as shown below.

```
<FRAMESET ROWS = "a,b">
<FRAME NAME = "name" SRC = "contents.html">
<FRAMESET COLS= "a,b">
<FRAME NAME= "name" SRC = "contents.html">
Continue defining for the columns here.
</FRAMESET>
You may continue defining more rows here with single columns.
</FRAMESET>
```

(see an example on pg 172 of recommended text).

To adjust a frame's margins

In the desired FRAME tag, type
MARGINWIDTH=w
MARGINHEIGHT=h

Where w,h are in pixels and w is space on left and right and h is space on top and bottom edges of the frame and the frame's contents.

Showing or hiding scroll Bars

Scrollbars can be set to be either there **all the time**, **never there at all** or **only there when needed**, i.e. when there is much text that requires scrolling.

To show them all the time, in the FRAME tag of the particular frame, type
SCROLLING=YES

To hide them all the time, type
SCROLLING=NO

Note that the default however is scrolling bars to appear only when needed. This can be represented by typing **SCROLLING = AUTO**, or simply type nothing at all.

Adjusting the color of borders

To adjust the color of all borders for a frameset, in the topmost FRAMESET tag, type
BORDERCOLOR=color

However, to change the color of rows, columns or individual frames, in the appropriate FRAMESET or FRAME tag, type
BORDERCOLOR=color

Adjusting Frame Borders

Inside the topmost FRAMESET tag, type
BORDER=n
Where n is in pixels and is the space between frames.

Keeping your visitors from resizing your frames

In the FRAME tag for the desired frame, type
NORESIZE

Targeting Links to Particular Frames

The initial content of a given frame is specified in the frameset page with the SCR tag. You can however have other pages appear in that same frame. This is done by adding a pointer called *target*, to the links to those pages.

To achieve this,

1. Ensure that the target frame has a name.
2. On the page where the link should appear, type

where *page.html* – the file to be displayed in the target frame
name- reference given to the target frame within the FRAME tag.

Nesting Frames

Framesets can be nested inside frames to achieve special effects. To nest Framesets,

1. build the child or inner frameset.
2. build the parent or outer frameset
3. Type SRC= "child.html" when you reach the frame in which you wish to nest the child frameset.

Here is an example:

```
<html><head><title>Nesting frames</title></head>
<frameset cols = "*,4,*">
<frame src = "bigindex.html">
<frame name= "main" src= "bcntourrc.html">
</frameset>
</html>
```

Note: for browsers that do not understand frames, use "noframes" tag ie type,
<noframes> just after the <frameset> tag, create the contents to appear if frames do not work, terminate with
</noframes></frameset>

LECTURE 12: FORMS

Forms enable your visitors to communicate with you. Some of the things you can perform with forms include among others:

- ❖ Get feedback
- ❖ Have a guestbook
- ❖ Take a survey
- ❖ See who is visiting you
- ❖ Sell stuff

Two basic parts of a form are:

1. Structure or shell-consists of fields, labels and buttons that visitors fill out
2. Processing script – takes that information and converts it into a format that you can read.

For the shell, you can create text boxes, radio buttons, password boxes, checkboxes, drop-down menus, larger text areas and clickable images. Each form element has a name that the server uses to identify the data once it is processed.

To process the form, you need a script either written in Perl or some other programming language (this may be learnt in Internet Programming module!).

CGI Scripts

CGI stands for Common Gateway Interface. It is the standardized way for sending information between the server and the script. A script on the other hand is simply a program written in a programming language as said before. Explain

CGI scripts have two functions:

1. They take all those name-value pairs separating them out into individual indelible pieces.
2. They do something with such data e.g. print them out, send email confirmation, store it on a server etc.

You can read more in texts.

Creating a form

On your html page, type

<FORM METHOD = POST ACTION= “script.url”> (there is also the GET method)

Create the form elements here

</FORM>

where *script.url* is the script to process the form elements.

A form element may look like this: <INPUT TYPE = “submit” VALUE = “Order Bed”>

Creating text boxes

Text boxes contain one line of free-form text e.g. for visitors to enter names, addresses etc. Type the following

<INPUT TYPE = “text” NAME = “name” VALUE= “value” SIZE= n MAXLENGTH = m>

where *text* – is as it is, *name* – text to identify the input data to the server, *value* – default data, *n*-width of box, *m* – maximum number of characters allowed in the box.

Note that Value, SIZE and MAXLENGTH are optional.

Creating Password Boxes

In such boxes, whatever is typed in is hidden by bullets or asterisks. Type the following

<INPUT TYPE = “password” NAME = “name”> e.g <INPUT TYPE=’password’ NAME=”Michael”

You can give optional values like SIZE and MAXLENGTH.

Creating Larger Text Areas

These allow visitors to enter more than one line of text. Useful for visitor responses like comments etc. They expand automatically as you enter more information. Type

```
<TEXTAREA NAME = “name” ROWS = n COLS = m WRAP>  
Type any default text if any but optional  
</TEXTAREA>
```

Where *n* = number of rows, default is 4, *m* = number of columns, default is 40 and *WRAP* to allow text to wrap. Note that ROWS, COLS and WRAP are optional.

Creating Radio Buttons

With these buttons, only one can be checked at a time. Type

```
<INPUT TYPE = “radio” NAME = “radioset” VALUE = “data” CHECKED>data
```

Where *radioset* identifies data sent to script and links the radio buttons together, *data* is the text sent to the server if a radio button is checked and *CHECKED* makes a particular radio button active by default.

Creating Checkboxes

These, unlike radio buttons, may be checked more than one at a time. They are linked by the value of the NAME attribute, just as in the case of radio buttons. Type

```
<INPUT TYPE = “checkbox” NAME = “boxset” VALUE = “value” CHECKED>value
```

Where *boxset* identifies data sent to script and links checkboxes together, *value* defines a value for each checkbox and *CHECKED* is as above.

Creating Menus

Menus provide your visitors with options to select from. Type

```
<SELECT NAME = “name” SIZE =N MULTIPLE>  
<OPTIONSELECTED VALUE = value”>  
Repeat the above line for as many options as you desire  
</SELECT>
```

Note: an example was given in class.

where *name* identifies data for the server, *n* is height (in lines) of menu the default is one, *MULTIPLE* allows visitors to select more than one. *SIZE* and *MULTIPLE* are optional. (hold down ctrl or command as you select).

Creating Submit button

Submit buttons are necessary for the visitor to deliver data filled on the form to you. Type

```
<INPUT TYPE = “submit” VALUE = “submit message”>
```

where *submit message* is the message to appear on the button itself. A default value of *Submit Query* will be there if

S. E. O. Ochingo, SCIT, JKUAT

you omit typing the VALUE part.

Creating the Reset Button

Reset button allows your visitors to start filling the form all over again in case they make mistakes. It is like rubbing out something already written. Type

`<INPUT TYPE = "reset" VALUE = "reset message">`

where *reset message* is the text to appear on the Reset Button.

Keeping Form Elements from being changed

This may be necessary especially where you do not want a visitor to change your default data. To achieve this, in the form element's tag, type, **READONLY** e.g.

`<TEXTAREA NAME= "votehistory" COLS = 25 ROWS = 3 READONLY>Woody on Monday, Cookie on Tuesday, Xixona on Sunday</TEXTAREA>`

You can as well do the following:

Set Tab Order: Allows your visitors to move from top to bottom of your form by simply pressing the Tab key. The order you give is followed. Type, in the form element's tag

TABINDEX = *n*, where *n* is a number between 1-32767.

For example: `<INPUT TYPE = "text" NAME = "firstname" TABINDEX = 3>`

Note: Tab order by default depends on the order of form elements.

Adding keyboard Shortcuts: This may not universally work as it is a new idea. However it may work for IE latest version.

Inside the form element's tag, type

ACCESSKEY = "keyboardshortcut"

For example: `<INPUT TYPE = "radio" NAME = "cats" ACCESSKEY= w>Woody (Alt-W)`

Disabling Form Elements: This may be necessary when you want to block your visitors from using certain parts of your form until certain conditions are met.

In the form element's tag, type **DISABLED**

LECTURE 13: AN INTRODUCTION TO CASCADING STYLE SHEETS – CSS

Cascading Style Sheets, CSS or simply Styles let you assign several properties at once to all the elements on your page marked with a particular tag. As an example, you can display all your H1 headers in a particular size, font and color. Although this may be achieved by HTML tags, Styles offer several advantages.

Advantages of using Style Sheets

1. They save time since several properties may be assigned at once to a tag.
2. They are easy to change i.e. you only need to make a change at one point and it applies for all.
3. Computers are better at applying styles than you are i.e. by use of tags, you may forget minor things like closing tags but computers won't.
4. They let you control tags in ways that are out of reach of HTML tags.
5. You can use styles with JavaScript to create dynamic effects known as Dynamic HTML or DHTML
6. They make it easy to create a common format for all your Web pages i.e. you only need to make changes to one central place and this is reflected in all your pages.

Disadvantages

1. No browser supports them completely.
2. You have to take time to learn them since they have their own distinct syntax.

The anatomy of a Style

A style is made up of:

- ❖ Tag name eg H1, P etc
- ❖ One or more definitions that determine how the elements marked with that tag should be displayed. Each definition contains a property, a colon, and one or more values. Here is an example.

H1 {font-size:12pt;color:red}

SETTING UP STYLE SHEETS

Styles can be applied in three main ways namely:

1. Locally.
2. Internally.
3. Externally.

You can use one or all the three methods and note that local methods override internal methods while internal methods override external methods.

1. Creating an Internal Style Sheet

Internal Styles are suitable for individual pages with lots of text. They let you set the styles at the top of your page that should be used throughout an HTML document. Type, between the <HEAD> and </HEAD> tags,

- <STYLE>
- Type name of tag eg H1
- type {define as many properties as desired for this tag, separating each property with a semicolon}
- repeat the above step for each tag.
- </STYLE>

An example may be found on page 244 of the recommended text.

Note: 1. You can define properties for several tags at once by separating each tag with a comma eg
H1,H2,H3{color:red}
2. To hide styles from browsers that do not yet understand them add comment tags `<!--` and `-->` after and before `<STYLE>` and `</STYLE>` tags. ie `<style><!-- ..... </style>`

2. Creating External Style Sheets

These are ideal for giving all the pages on your Web site a common look. Each page is set to consult the external sheet thereby ensuring that each page has the same setting. Type

- Create a new text document and type the name of the tag eg H1, whose properties you wish to define.
- Type {define as many properties as desired like in the previous case above}
- Repeat the above step as many times as you desire.
- Save the document as a text only format with an extension **.css** (for Cascading Style Sheet), in the desired directory.

Using External Style Sheets

Once created, link it to each page it should format. Type in the HEAD section of each HTML page in which you wish to use the style sheet

`<LINK REL=stylesheet TYPE= "text/css" HREF= "url.css">`

where *url.css* is the name of the style sheet saved.

Applying Styles locally

This is a small-scale way and a beginning to applying style sheets. Type within the HTML tag that you want to format,

STYLE= "property:value;property:value;..."

eg `<p style = "background:yellow; color:red; font-weight:900">`

See an example on pg 248 of recommended text.

Defining Styles for Classes

You can divide your HTML elements into categories or classes in order to apply styles to them separately. An example is creating a class of introductory paragraphs that will have slightly different formatting than regular paragraphs. Type

- Mark the elements in your HTML page that belongs to the class by adding

CLASS=classname

where *classname* is the identifying word for the class

- In the STYLE section of your HTML page, type
Parenttag.classname

Where *parenttag* is the tag that the class is subset of.

- Type {property:value;...}

See pg 249.

Identifying Particular Tags

This is an alternative to creating a whole class of HTML tags. You simply identify individual tags, then either apply style sheet information or JavaScript functions to them. Type

- Identify the element on your page by adding **ID=idname** (where *idname* is the identifying word for the tag)
- In the STYLE section of your HTML page type **parenttag#idname** where *parenttag* is the kind of HTML tag that is identified with the ID attribute.
- Type {add usual data as before}

Creating Custom HTML tags

Two generic tags, DIV and SPAN in HTML 4 can now be combined with classes and ID in order to create custom HTML tags. DIV is for block-level elements while SPAN is for inline elements.

Creating Custom Block-level HTML Tags

This can be done by adding a class or ID to the DIV tag and then define the styles that it should apply. In the STYLE section or in an External style sheet, type

- DIV.className, where the *classname* identifies the class you are going to use.
- Or
- DIV#IDname, where *IDname* is the identifying word for this particular instance of the DIV tag.
 - Type { add usual information}

Using Custom Block-level Tags

This is similar to using other HTML tags only that you have to specify the class or ID. At the beginning of the desired section of your document type,

<DIV CLASS= "classname">

or

<DIV ID= "IDname">

- Create the contents of this section
- Close the tag </DIV>

Creating Custom inline Tags

They are useful for combining a set of text formatting characteristics in one tag. In the STYLE section or in an external style sheet, type

SPAN.className

Or

SPAN#idname

Where *classname* is the identifying word for the class you are going to use and *idname* identifies this particular text.

- Type {add usual information}

Using Custom Inline Tags

At the beginning of the desired words, type

or

Create the text you wish to affect

Close

Defining Styles for Links

Link styles have a special syntax. This can allow you to change the background and foreground color of your links to make them stand out if you do not like the underlined links. Type

A:link- This changes the appearance of links that have not yet been or currently aren't being clicked or pointed at. For example

A:link{background:yellow;color:green;text-decoration:none}

A:visited – changes the appearance of links that the visitor has already clicked.

A:active –changes the appearance of links when clicked.

A:hover- changes the appearance of links when pointed to. *(not supported by Netsape communicator 4)*

Note: typing just A:{property:value} changes the appearance of all the links at once.

FORMATTING TEXT WITH STYLES

The W3 Consortium is hoping to deprecate several tags e.g. FONT tag and instead make use of styles. This may result in saving you both in time and typing.

How to choose a font family

You can specify more than one font. Type

font-family:familyname

e.g. H1 {font-family: "Nueva Roman", Lithos Regular"}

Creating Italics and Bold

For italics, type

font-style:italic and to stop it type **font-style:normal**

e.g. P {font-style:italic}

P.intro {font-style:normal}

For bold, type **font-weight:bold** and to stop type **font-weight:normal**

Note: You can use bolder or lighter to use a value relative to the current weight or multiples of 100 between 100 to 900.

Setting font size

Use font-size:size e.g. H1 {font-size:20pt}

There are however many ways of giving font size which can be learnt from the recommended text.

Setting line height

Line height refers to the space between each line in a paragraph. Type

Line-height:n where *n* is a value in pixels, points, or a number to be multiplied by the font size to obtain the desired line height. There are other ways too.

Setting text color

Type

color:colorname e.g. H1 {color:yellow}

Changing text's background

This refers to the background of the tagged text and not to the entire page. Type

background:color or **background:transparent**

You can use an image for background by adding **url(image.gif)**. If you want to tile the image, add the word **repeat**.

To align text

Certain HTML tags can always be set to be aligned left, center or justified. Type **text-align:direction** where *direction* can be left, right, center or justify.

To underline, overline or strike through text

Type,

text-decoration:underline

text-decoration:overline

text-decoration:line-through

Making text to blink

text-decoration:blink

Changing the text case

text-transform:capitalize – this puts first character capital.

text-transform:uppercase – changes all letters to uppercase.

text-transform:lowercase – changes all letters to lower case.

text-transform:none- leaves text as is.

LECTURE 14: GENERAL TROUBLESHOOTING

Helping when your page refuses to work

This lecture tells us some of the common html errors and some of which are of our own making.

- i. First and foremost, check to ensure that your code is correct eg are the spellings for tags correct.
- ii. If the browser displays your code, check to ensure that:
 - Is document saved as text only, with extension .html or .htm
 - Have you began your page with <html>
 - Is Windows adding .txt extension to your file names? Check and correct accordingly.
- iii. Your page is ok in one browser and horrible in another

- It could be that you have ignored terminating some tags that different browsers interpret differently ie some browsers may require that you have to terminate tags officially.
- Avoid too much use of proprietary tags.
- iv. Images do not appear
 - Ensure file names are correct
 - Ensure image path is correct
 - Ensure image is saved in the correct format eg .gif, .jpg etc

All in all, if you cannot come out of your problem, try and use an htm validator. This will try to point out problems on your page. Also do not be alone. Consult as well.

Publishing your web page on the web

Publishing your page on the web involves transferring your pages to your web host server so that people can get to them. Your web host can be an Internet service Provider (ISP).

Testing your page

Before loading your files on the web, you need to test them first to ensure all is well. This is achieved by opening your browser and testing as you have been doing in the lab. If all does not please you, open the code in any text editor and correct any errors. Try reloading the page in a browser until you are satisfied. You do not have to close the code file while testing your pages on the browser.

After this is done, transfer your files to the server and change file permissions if you have not yet done so. Now type your url and see.

Finding a host for your site

There are lots of companies ready to host your pages. Consider the following factors when deciding on a host

- i. How much disk space will they let you, do not pay for more space that you need.
- ii. Check if they offer any technical support and especially how fast enough
- iii. Is it possible that they can register a domain name for you and at how much.
- iv. How fast is their Internet connection
- v. Can they offer you dial-up access to the Internet
- vi. Do they offer web hit statistics to let you know how many people hit your page
- vii. Can they allow other features like CGI scripts, server Side Includes etc to be run

How to get your own domain name

By registering it at a fee. Your web host can then create a virtual domain on their sever with your domain name. This has a particular advantage that if you decide to change a web host, all your links will continue to work.

Transferring files to the file server

You need programs like WS-FTP for transferring files in the correct format to servers.

Getting people to visit your page

Because of the huge number of web pages available today, you may need to shout a little harder to be seen out there. This section outlines some of the tags that may make you be noticed by search engines.

Helping visitors find your page

In the HEAD section of your page, type

```
<META NAME="keywords" CONTENT="someword">
```

Where **someword** are a few words describing exactly the topic discussed on your page. Separate each word with a coma and a space. An example looks like this

```
<META NAME="keywords" CONTENT="alpaca, alpacas, wool, lama, llama">
```

Controlling your page's summary

This summary allows your visitor to your page to have a preview of the summary of your page. They may then from here decide to either go in or not. The more attractive your summary is, the better the chance of it being viewed. Type

`<META NAME= "description" CONTENT="somesentence">` where **somesentence** is a concise sentence or two that describe your page and hopefully persuades your folks to click it.

Keeping search engines away from noticing your page

This may happen like in the case where you feel that your page is personal. Type in the HEAD section of your page `<META NAME= "robots" CONTENT= "NOINDEX, NOFOLLOW">`

Html EDITORS

You can use any text editor however there are dedicated editors which are simply point-and-shoot programs. They are more expensive and generally less flexible. Below is some information.

What html editors offer	Disadvantages of html editors
● Insert opening and closing tags with a single click ● Check and verify syntax in html and typos ● Allow you to add attributes by clicking buttons instead of typing words in a certain order in a certain place in a document ● Offer varying degrees of WYSIWYG display of your page ● Correct mistakes in existing html pages ● Make it easy to use special characters	● Do not all recognize new or non-standard html codes ● Do not all support forms, frames and tables ● Are more difficult to learn, less intuitive than they promise ● Cost money unlike simple editors normally packaged free with system software ● Use more space on disk and more memory ● Some add proprietary information, like their names, and tags to the html document ● Some eliminate tags that they do not understand, even if the tags are part of the standard html specs.

LECTURE 15: JAVA APPLETS

- All **applets** are subclasses of **Applet**, and thus must all import **java.applet**. They must also import **java.awt** ie Abstract Window Toolkit.
- They are not executed by the console-based java runtime interpreter but by a web browser or an applet viewer.
- Execution of an applet does not begin at `main()` and output not performed by **System.out.println()**.
- Once an applet is compiled, it is included in a **html** file using the **applet** tag.
- Applet is executed when the java-enabled browser encounters the **applet** tag
- To view and test an applet more conveniently, simply include a comment at the head of your Java source file that contains the **applet** tag as shown below.

```
/*  
<applet code = "MyApplet" width=200 height=60></applet>  
*/
```

NB:

- `MyApplet` is the applet name
- Width, height specify the size of display in pixels.

An Applet skeleton

An applet must override several methods, five of the basic include **init()**, **start()**, **stop()**, **destroy()** – all defined by applet and **paint()** – defined by AWT component class. These five methods can be assembled into the skeleton shown here (see attached code no. 1).

NB: the skeleton does nothing but can be compiled and run.

The order in which methods are called by AWT is as below:

- i. `init()`
- ii. `start()`
- iii. `paint()`

Then on terminating the applet

- iv. `stop()`
- v. `destroy()`

Here are the explanations:

`init()` – is the first method to be called and is where you should initialize variables. It is called only once during the run time of your applet.

`start()` – called after `init()`. Also called to restart an applet after it has been stopped. It is called each time an Applet's html document is displayed on screen, so if a user leaves a page and comes back, the applet resumes execution at `start()`.

`paint()` – is called each time your applet's output must be redrawn. `paint()` is also called when the applet begins execution and has one parameter of type **Graphics**.

`stop()` – is called when a browser leaves the html document containing the applet, e.g. when it goes to another page.

`destroy()` – is called when the environment determines that your applet needs to be removed completely from memory. `stop()` is always called before `destroy()`.

Defn: Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically executed, and run as part of a web document.

On arrival, it has limited access to resources so it poses no danger to the client.

Here is a simple applet:

```
import java.awt.*;  
import java.applet.*;  
  
public class SimpleApplet extends Applet{  
    public void paint(Graphics g){
```

```

        g.drawString("A Simple Applet",20,20);
    }
}

```

This example above causes the statement "A Simple Applet" to be displayed at location 20,20 on the screen. All applets must extend the Applet i.e. must be a subclass of Applet.

Compiling and running an applet

- Enter the source code e.g. for "SimpleApplet" above
- Compile in the usual way of compiling java programs
- To run it, there are two ways as discussed here below:
 - Executing the applet within a **java-compatible web browser** e.g. Netscape Navigator
 - Using an applet viewer e.g. JDK tool, **appletviewer**.

To use the first method above, you write a short **html** text file that contains the appropriate **APPLET** tag e.g.

```

<applet code = "SimpleApplet" width=200 height=60>
</applet>

```

After you create this file, you can execute your browser and then load this file which then causes SimpleApplet to be executed.

To use the second method above, i.e. appletviewer, you may also execute the html file as above e.g. if the file is RunApplet.html, then the following command will run **SimpleApplet**.

C:\>appletviewer RunApplet.html

LABS

ICS 2203: LAB

The following exercises are intended to help us to understand what we covered in lecture 3, 4 and 5. Part of it is meant to cater for the coming lecture next week.

Exercise 1:

Using the general structure given out in the last lecture, create a simple Web page. Make your document to be at least three paragraphs and try to apply the tags learnt so far. Save this file and try viewing it. The content of this file can be a description of you.

Exercise 2:

Create a second Web page that is almost the same size as in exercise 1. Apply the tags as much as you can. Let this page be some description of your University. Save it and view it accordingly.

Exercise 3:

Create a normal MS-Word file (i.e with .doc extension) and save it in the same folder as all the rest. (Note: this will be used after this stage)

ICS 2203

Lab: Cascading Style Sheets

1. Applying styles locally

Please type in the following and see what happens.

```
<HTML><HEAD><TITLE>Applying Styles Locally</TITLE></HEAD>
<BODY>
<P STYLE= "background:yellow;color:red;text-transform:capitalize;border:double medium green;font-
weight:900;padding:0.5em">On this page, you will learn a little about each of our cats, and how they have
created their own style.
<H1>Llumi, the Huntress </H1>
<P>Llumi is our sweet, but <EM>ferocious</EM> hunter-kitty. Maybe it's because she was born out in the
wild or maybe because she was an orphan.
<BODY> <HTML>
```

2. Defining Styles for Classes

```
<HTML><HEAD><TITLE>Applying Styles Locally</TITLE>
<STYLE>
H1 {text-align:center;letter-spacing:.5em;background:green; color:yellow; font:normal 20pt "Nueva Roman",
"Lithos Regular"}
P {text-align:justify;text-indent:8pt;font:10pt/15pt "Myriad Roman", "Verdana"}
P.intro {text-indent:0;font:italic bold}
P.intro EM {font-style:normal}
</STYLE>
</HEAD>
<BODY>
<P CLASS=intro> on this page, you'll learn a little about each of our cats, and how they have created
<EM>their own</EM>style.
<H1>Llumi, the Huntress </H1>
<P>Llumi is our sweet, but <EM>ferocious</EM> hunter-kitty. Maybe it's because she was born out in the
wild or maybe because she was an orphan.
```

3. Identifying Particular Tags

```
<HTML><HEAD><TITLE>Applying Styles Locally</TITLE>
<STYLE>
H1 {text-align:center;letter-spacing:.5em;background:green; color:yellow; font:normal 20pt "Nueva Roman",
"Lithos Regular"}
P {text-align:justify;text-indent:8pt;font:10pt/15pt "Myriad Roman", "Verdana"}
P#intro {text-indent:0;font:italic bold}
P.intro EM {font-style:normal}
</STYLE>
</HEAD>
<BODY>
<P ID=intro>On this page, you'll learn a little about each of our cats, and how they have created <EM>their
own</EM>style.
<H1>Llumi, the Huntress </H1>
<P>Llumi is our sweet, but <EM>ferocious</EM> hunter-kitty. Maybe it's because she was born out in the
wild or maybe because she was an orphan.
```

4. Creating and Applying an External Style Sheet.

Create this file and save it as a text file with .css extension.

```
H1 {text-align:center;letter-spacing:.5em;background:green;color:yellow;font:normal 20pt "Nueva Roman",
"Lithos Regular"}
P {text-align:justify; text-indent:8pt; font:10pt/15pt "Myriad Roman", Verdana"}
P.intro {text-indent:0;font: italic bold}
```

```
P.intro EM {font-style:normal}
A:link {background:yellow;color:green;text-decoration:none}
A:visited {background:orange;color:yellow;text-decoration:none}
A:hover {background:red; color:white;text-decoration:none}
```

```
A:active {background:white;color:black;text-decoration:none}
```

In order to link the file, do the following:

```
<HTML><HEAD><TITLE>Creating An External Style Sheet</TITLE>

<LINK REL= stylesheet TYPE= "text/css" HREF= "style.css">
</HEAD>
<BODY>
<P class=intro>on this page, you will learn a little about each of our cats, and how they have created their own
style

<H1>Cookie, the invisible</H1>

<P>When we lived in Barcelona, people would come to our house and rave about our big, American cat, <A
HREF= "woody.html">Woody</A>. Poor Cookie never even got a mention. Of course, that was because as
soon as she had the doorbell, she ran under the bed and stayed there until the guests had long gone home. I am
not sure why she was not shy with people. It might have to do with the fact that she was born in a planter on the
busiest street in Barcelona and I abducted her.
```

Note:

- **style.css** represents the name you used to save your own .css text file.
- **woody.html** represents a .html file you have on your system.

5. Now have ago at your own creations. Explore as much as you can.

ICS 2203 LAB

1. From the previous lab sheet, i.e. lab 2, you had the opportunity to create two. Html files and one doc document. Using those files, try and link the three documents together so that one is able to jump from your first page, i.e. the page describing you up to the last, i.e. the doc document.

Assuming for now that this is your official web site, embed your photograph on your first page and let your description wrap around your image.

Try as well to utilize latest tags learnt so far e.g. linking icons to external images or documents, background color and backing images etc.

2. Create some long html document. The text can come from paragraphs of a daily Newspaper. Show how you would use images to label links to other docs you have. Show the techniques of creating anchors effectively.
3. Using the techniques of lists, show how you would create things like ordered and unordered list, nested lists, definition lists. etc.

S. E. O. Ochingo, SCIT, JKUAT

SCS 218
Lab 3
Groups A & B

4. From the previous lab sheet, ie lab 2, you had the opportunity to create two. Html files and one doc document. Using those files, try and link the three documents together so that one is able to jump from your first page, ie the page describing you up to the last, ie the doc document.

Assuming for now that this is your official web site, embed your photograph on your first page and let your description wrap around your image.

Try as well to utilize latest tags learnt so far eg linking icons to external images or documents, background color and backing images etc.

5. Create some long html document. The text can come from paragraphs of a daily Newspaper. Show how you would use images to label links to other docs you have. Show the techniques of creating anchors effectively.

3. Using the techniques of lists, show how you would create things like ordered and unordered list, nested lists, definition lists. etc.

ICS 2203: LAB

The following exercises are intended to help us to understand what we covered in lecture 3, 4 and 5. Part of it is meant to cater for the coming lecture next week.

Exercise 1:

Using the general structure given out in the last lecture, create a simple Web page. Make your document to be at least three paragraphs and try to apply the tags learnt so far. Save this file and try viewing it. The content of this file can be a description of you.

Exercise 2:

Create a second Web page that is almost the same size as in exercise 1. Apply the tags as much as you can. Let this page be some description of your University. Save it and view it accordingly.

Exercise 3:

Create a normal MS-Word file (i.e with .doc extension) and save it in the same folder as all the rest. (Note: this will be used after this stage)